

Apuntes - Aprendizaje Automático para Datos en Grafos

Contents

Clase 1 – Introducción y motivación	2
Organización del curso	2
¿Qué es un grafo?	2
Motivación: impacto de las redes	2
Enfoque del curso	2
Señales en grafos	3
De la estructura de la señal a la Transformada de Fourier en grafos	3
Modos de frecuencia del Laplaciano	4
Clase 2 – Modelos Estadísticos de Grafos	4
Motivación: ¿para qué modelar estadísticamente grafos?	4
Modelo de Grafos Aleatorios (marco general)	4
Grafos Aleatorios Clásicos: Erdős–Rényi(-Gilbert)	4
Configuration Models	5
Modelo Small World (Watts-Strogatz)	6
Modelos de Variables Latentes	7
Random Dot Product Graphs (RDPG)	9
Grafos Geométricos (RGG)	9
Geometría Hiperbólica y Curvatura	10
Resumen y moralejas (tabla del profesor — repaso rápido)	10
Clase 3 – Inferencia de grafos	11
Objetivo	11
Los 3 problemas de inferencia de grafos	11
Association Networks	12
Undirected Gaussian Graphical Models	13
Aprendiendo grafos a partir de observaciones de señales suaves	14
* Posibles puntos de examen	16
Clase 4 – Graph Neural Networks (GNNs)	16
Intro	16
Motivación: Empirical Risk Minimization (ERM)	17
Convoluciones en grafos	17
GNN: Construcción	18
Múltiples features	18
Entrenando una GNN	19
Permutation equivariance	20
Perturbaciones	20
* Posibles puntos de examen	20
[!] Puntos poco claros / a revisar	20

Clase 1 – Introducción y motivación

Aprendizaje Automático para Datos en Grafos Docentes: Marcelo Fiori (IMERL, UDELAR – Universidad de la República) y Federico Larroca Contenido del curso basado en el material de Gonzalo Mateos (University of Rochester)

Apuntes tomados en clase, complementados con `clase_introduccion.pdf`. Cubren la clase hasta la hoja 54 inclusive, con la excepción de las diapositivas de “**Covarianzas y componentes principales**”, que se saltaron.

Organización del curso

- 5 clases + 4 notebooks exploratorios (los notebooks no son obligatorios, son herramientas de apoyo)
 - **Aprobación:** cuestionario final de ≈ 10 preguntas de opción múltiple (mínimo 6 correctas)
-

¿Qué es un grafo?

Según la RAE, un grafo es “*una colección de cosas interconectadas*”: hay múltiples cosas (nodos) y estas están conectadas (aristas) — dos extremos.

Contexto histórico

- **1735** — L. Euler funda la teoría de grafos: los siete puentes de Königsberg
- **1845** — Leyes de los circuitos eléctricos (Kirchhoff)
- **1874** — Estructura molecular en química (Cayley)
- **1930** — Representación en red de interacciones sociales
- Redes eléctricas de potencia; telecomunicaciones e Internet
- Google, Facebook, Twitter

Entender sistemas complejos $\Leftrightarrow$ entender las redes detrás de ellos.

Motivación: impacto de las redes

- **Salud:** predicción de epidemias; el Proyecto Conectoma Humano (*Human Connectome Project*) para mapear la circuitería cerebral.
- **Seguridad nacional:** el análisis de redes sociales fue clave para capturar a Saddam Hussein.
- **Descubrimiento científico:** el plegamiento de proteínas (predecir la estructura 3D de una proteína a partir de su secuencia 1D de aminoácidos).

Objetivos de la ciencia de datos en redes

- **Revelar** patrones y propiedades estadísticas de los datos de red
- **Entender** los fundamentos del comportamiento y la estructura de las redes
- **Diseñar** redes más eficientes en el uso de recursos, robustas y socialmente inteligentes

Inferencia estadística $\approx$ aprendizaje automático: la teoría de grafos se encuentra con la inferencia estadística.

Enfoque del curso

Procesamiento de señales y aprendizaje a partir de datos de red.

Un desafío central es que estos datos no tienen fuertes *priors* estructurales ni geométricos (a diferencia de, por ejemplo, imágenes o señales temporales regulares).

¿Qué se puede hacer con estos datos?

- Visualización de grafos y descubrimiento de patrones
- Modelado y generación de grafos
- Clustering y detección de comunidades
- Predicción de enlaces (aristas)
- Clasificación de nodos y aprendizaje semisupervisado
- Clasificación de grafos

Todo esto se plantea tanto para grafos dirigidos como no dirigidos (que además pueden ser dinámicos). En ambos casos los grafos se representan mediante matrices — la matriz de adyacencia es simétrica en el caso no dirigido, lo cual resulta muy útil. Los métodos para trabajar con estos datos pueden ser **probabilísticos** o **determinísticos**, y en general se apoyan en la estructura (el grafo) subyacente.

Señales en grafos

Una **señal** es un atributo asociado a cada nodo del grafo (por ejemplo, una etiqueta binaria del vértice). Se representa como un vector sobre la cantidad de nodos (o como una matriz, si se trata de una señal temporal).

Las aristas también pueden tener atributos, pero eso no es el enfoque de este curso.

Pregunta abierta (anotada en clase): ¿por qué trabajar con señales?

Procesamiento de señales en grafos (GSP)

- Grafo G con matriz de adyacencia $A \in \mathbb{R}^{N \times N}$, donde A_{ij} representa la proximidad entre los nodos i y j .
 - Señal $x \in \mathbb{R}^N$ sobre el grafo, donde x_i es el valor de la señal en el nodo i .
 - Idea central del GSP: **explotar la estructura codificada en A** para procesar x .
-

De la estructura de la señal a la Transformada de Fourier en grafos

El procesamiento de señales consiste, en esencia, en explotar la estructura de la señal.

Ejemplo — el tiempo discreto como grafo cíclico:

- El grafo es un ciclo, representado por una matriz **circulante** (unos en la diagonal desplazada, $A_{i,i+1}$).
- El tiempo n sigue al tiempo $n-1 \Rightarrow$ el valor x_n es similar a $x_{n-1} \Rightarrow$ se espera que la señal sea periódica (suave).
- Calculando los autovectores de la matriz circulante se obtienen las **bases de Fourier**.
- Hacer la transformada de Fourier es, en el fondo, hacer un **cambio de base** del vector señal.

Generalización — Transformada de Fourier en Grafos (GFT):

- Se define un operador de desplazamiento en el grafo S (puede ser la matriz de adyacencia, el Laplaciano, u otro), con descomposición en autovectores $S = V\Lambda V^{-1}$ (se apoya en el teorema espectral).
 - **GFT (Graph Fourier Transform):** proyección de la señal sobre el espacio de autovectores del operador de desplazamiento S .
-

Modos de frecuencia del Laplaciano

La **variación total** de la señal x con respecto al Laplaciano L :

$$TV(x) = x^\top Lx = \sum_{i,j} A_{ij}(x_i - x_j)^2$$

Es una medida de **suavidad** de la señal sobre el grafo G (energía de Dirichlet): cuanto más parecidos son los valores de x entre nodos conectados, menor es $TV(x)$.

ewpage

Clase 2 – Modelos Estadísticos de Grafos

Apuntes tomados en clase, completados y verificados contra clase_2.pdf (69 diapositivas). Formato pensado para repaso rápido de cara al multiple choice: lo importante en negrita, fórmulas clave, ejemplos solo cuando ayudan a fijar el concepto.

Motivación: ¿para qué modelar estadísticamente grafos?

1. **Explicar mecanismos** detrás de propiedades observadas en grafos reales (ej: efectos *small-world*, distribuciones de grado con ley de potencia)
 2. **Modelo nulo** para testear propiedades de un grafo (ej: ¿es raro el clustering coefficient que observo?)
 3. **Evaluar factores predictivos** de relaciones entre nodos (ej: ¿hay efectos transitivos o de reciprocidad?)
 4. **Generación de grafos** para evaluar algoritmos en condiciones controladas
-

Modelo de Grafos Aleatorios (marco general)

Colección $\{P(G), G \in \mathcal{G} : \theta \in \Theta\}$:

- $\mathcal{G}$: conjunto de grafos posibles bajo el modelo
- $P(\cdot)$: distribución de probabilidad sobre $\mathcal{G}$
- θ : vector de parámetros, $\theta \in \Theta$

La riqueza y utilidad del modelo depende de cómo se especifique $P(\cdot)$.

Especificación del modelo (de lo simple a lo complejo)

1. $P(\cdot)$ **uniforme** en $\mathcal{G}$, respetando restricciones estructurales — ej: Erdős-Rényi, Configuration model
2. Inducir $P(\cdot)$ mediante un **modelo generativo** — ej: small world, preferential attachment, copying models, deep generative models
3. Modelar **propiedades estructurales** y su efecto sobre la topología de $\mathcal{G}$ — ej: *exponential random graph models*
4. Modelar la tendencia a conectarse mediante **variables latentes** — ej: SBM, grafones, RDPGs

El costo computacional de los algoritmos de inferencia y generación es un aspecto importante a la hora de elegir.

Grafos Aleatorios Clásicos: Erdős-Rényi(-Gilbert)

Se asigna la misma probabilidad a todos los grafos no dirigidos de cierto orden y tamaño.

- Colección $\mathcal{G}_{N_v, N_e}$: grafos $G(V, E)$ con $|V| = N_v$, $|E| = N_e$. $P(G) = \binom{N}{N_e}^{-1}$ para cada G , con $N = \binom{N_v}{2}$.

- **ER(n, p)** (o $G_{n,p}$): caso particular no dirigido, n nodos, cada arista (u, v) existe con probabilidad p independiente del resto.
- **Simulación eficiente:** sortear $\binom{n}{2}$ Bernoulli(p) i.i.d. es ineficiente si $p \ll n^{-1}$ (grafo esparso, mayoría ceros). Mejor: sortear Geo(p) i.i.d. para decidir qué aristas generar $\rightarrow$ corre en $O(N_v + N_e)$.

Propiedades de ER(n, p)

P1) Distribución de grados:

$$D_i = \sum_{j \neq i} A_{ij}, \quad (A_{ij})_{j \neq i} \sim \text{Ber}(p) \text{ i.i.d.} \Rightarrow D_i \sim \text{Bin}(n-1, p)$$

$P(d)$ está concentrado alrededor de $p(n-1)$ con colas exponenciales — cota de **Hoeffding**:

$$P(|D_i - p(n-1)| \geq t) \leq e^{-2t^2/(n-1)}$$

¿Por qué importa esto? La Binomial es una distribución de **cola liviana**: la probabilidad de un nodo con grado muy distinto al promedio cae exponencialmente rápido (por Hoeffding). Es decir, en ER **todos los nodos tienen un grado parecido**. Esto contrasta fuerte con las redes reales, que suelen tener **colas pesadas** (ver nota en Configuration Models) — es una de las razones por las que ER no las modela bien.

P2) Transición de fase en la aparición de una componente gigante:

- $np > 1 \rightarrow$ componente gigante de tamaño $O(n)$ w.h.p.
- $np < 1 \rightarrow$ componentes de tamaño $O(\log n)$ w.h.p.

Demostración (idea): proceso de exploración con estados activo (A_t), inactivo (I_t), explorado (E_t). El par (E_t, A_t) es una cadena de Markov a tiempo continuo. Reescalando con $p = c/n$ (vecinos promedio $\rightarrow c$) y tomando $n \rightarrow \infty$ (límite fluido), se llega a una ecuación diferencial con solución analítica:

$$e_\infty = \mu, \quad a_\infty = 1 - \mu - e^{-c}$$

Hay componente gigante $\iff c > 1$, y su tamaño (proporcional a n) es la solución positiva de $1 - \mu = e^{-c}$.

P3) Clustering coefficient pequeño, de orden $O(n^{-1})$, y diámetro corto, de orden $O(\log n)$ w.h.p.

¿Qué es el clustering coefficient? Mide qué tan seguido los vecinos de un nodo también son vecinos entre sí (proporción de “triángulos cerrados” sobre tríos conectados). *Interpretación:* alto $\rightarrow$ los amigos de tus amigos también son tus amigos (típico en redes sociales reales, por homofilia). En ER, $cl(G) = O(n^{-1}) \rightarrow 0$: casi no hay triángulos, porque cada arista se sortea de forma puramente independiente y al azar, sin ninguna noción de “cercanía” entre nodos.

¿Qué es el diámetro de un grafo? Es la mayor de las distancias más cortas entre cualquier par de nodos — el “peor caso” para llegar de un nodo a otro. *Interpretación:* chico $\rightarrow$ cualquier par de nodos está a pocos saltos (efecto “mundo pequeño”). En ER, $diam(G) = O(\log n)$: aunque las conexiones son puramente aleatorias, el grafo ya es “pequeño” — el problema de ER no es el diámetro, es que **le falta clustering**.

Esta combinación (diámetro chico pero clustering $\rightarrow 0$) es precisamente lo que **no** coincide con redes reales, que tienen ambas cosas a la vez — motiva Configuration models y luego Watts-Strogatz.

Configuration Models

Receta para generalizar ER: especificar $\mathcal{G}$ con grafos de orden N_v que además cumplan cierta característica, y asignar probabilidad uniforme dentro de esa colección.

- Si solo se conoce la cantidad media de vecinos $c \rightarrow \text{ER}(n, c/n)$ es razonable.
- Si se conoce la **secuencia o distribución de grados completa** $\rightarrow$ hace falta el **configuration model**.

¿Qué es una distribución de grado de “cola pesada” (heavy-tailed)? El profesor insiste bastante en esto, así que vale la pena tenerlo claro. Es lo opuesto a la Binomial de ER: la probabilidad de nodos con grado muy alto **no** decae exponencialmente, sino mucho más lento — típicamente como una **ley de potencia (power-law)**: $P(d) \propto d^{-\tau}$. *Interpretación práctica*: en vez de que todos los nodos se parezcan (como en ER), aparecen unos pocos nodos “hub” con muchísimas más conexiones que el resto — pensá en aeropuertos muy conectados, sitios web muy enlazados, o actores que trabajaron en cientos de películas. Casi todas las redes reales (Internet, redes sociales, la WWW, colaboración de actores) muestran este fenómeno, y **ER no lo puede generar** (su distribución de grado siempre está concentrada, con colas exponenciales). Esta es la motivación central del configuration model: dado que ER falla en esto, se necesita un modelo que permita *fijar directamente* la distribución de grado deseada (heavy-tailed o la que sea).

Ejemplos empíricos con colas pesadas mencionados en clase: conectividad entre Autonomous Systems, actores compartiendo películas en IMDb, amigos en Facebook, la WWW.

Definición: secuencia de grados dada $d = \{d_1, \dots, d_{N_v}\}$. El tamaño queda determinado: $N_e = \sum_i d_i / 2$. Equivale a especificar el modelo como distribución condicional sobre $\mathcal{G}_{N_v, N_e}$.

- Útiles como modelos **nulos** (comparar grafo observado vs. uno con $P(d)$ power-law) y para imponer restricciones de conectividad (ej: sin nodos aislados).

Simulación — $\text{CM}_n(d)$

- **Matching algorithm** (usado para el análisis): nodos con semi-aristas (“stubs”), se unen al azar.
 - No toda secuencia de grados es “gráfica”; pueden aparecer self-loops y multi-aristas $\rightarrow$ se borran.
 - Se puede probar que, bajo condiciones (ej. media finita), la distribución resultante $P^{(er)}(d)$ converge a $P(d): \sum_d |P^{(er)}(d) - P(d)| \rightarrow 0$.
 - Además, $\text{CM}_n(d)$ genera al azar uniforme entre todos los grafos con esa secuencia de grados.
- **Switching algorithm**: parte de un grafo inicial y aleatoriamente intercambia pares de aristas, repitiendo $\sim 100N_e$ veces.

Resultados de $\text{CM}_n(d)$

1. **Transición de fase** en la aparición de componente gigante, tamaño $O(n)$. Condición: $\kappa = E\{D(D-1)\}/E\{D\} > 1$ (en ER equivale a c , igual que antes). Intuición: el grado (menos uno) de un vecino típico debe superar 1 en promedio — se usa la distribución *size-biased* del grado del vecino.
2. **Clustering coefficient** $\rightarrow 0$, igual que en ER.
3. **Distancias típicas** dependen de $P(d)$:
 - Si $\text{Var}(D) < \infty \rightarrow$ distancias típicas $O(\log n)$
 - Power-law $P(d) \propto d^{-\tau}$ con $\tau \in (2, 3)$ (media finita, varianza infinita, es decir, **cola pesada de verdad**) $\rightarrow$ distancias típicas $O(\log \log n)$, aún más chico que en el caso ER-like
4. Se pueden acotar el *independence number* de estos grafos analizando procesos de exploración mediante límites fluidos (Brightwell-Janson-Luczak’17; Bermolen-Jonckheere-Moyal’17).

Modelo Small World (Watts-Strogatz)

Contexto histórico

- “Seis grados de separación” se popularizó con la obra de teatro de [Guare’90], pero el concepto es más viejo.
- **Karinthy (1929)**: cuento húngaro — el mundo se “achica” por el aumento de conectividad humana; apuesta a poder contactar a cualquiera con ≤ 5 intermediarios.
- **Kochen-Pool (1950s)**: primer tratamiento matemático formal, pero deja sin responder la cuestión de los “grados de separación”.
- **Milgram (1967)**: experimento que sí lo mide.

Experimento de Milgram

- Pregunta: ¿cuál es la distancia típica entre dos personas en la red social global?
- Método: 296 cartas enviadas desde Wichita (Kansas) y Omaha (Nebraska) a un único contacto en Boston. Regla 1: si el destinatario actual conoce al contacto por su nombre de pila, se la manda directo. Regla 2: si no, la reenvía a quien crea más cercano al contacto.
- **Resultado:** 64 de 296 cartas llegaron, con largo promedio $\mu = 6.2 \rightarrow$ inspiró la obra de Guare.
- **Conclusión:** caminos cortos conectan a dos personas cualesquiera, y en abundancia.

¿Es razonable la teoría del mundo pequeño?

Si cada persona tiene 100 amigos, cada uno con otros 100... tras 5 grados hay 10^{10} personas (más del doble de la población mundial) — **no es un modelo razonable** para una red social real, que típicamente tiene:

- **Homofilia** [Lazarsfeld'54]
- **Triángulos cerrados abundantes** [Rapoport'53]

Pregunta clave: ¿cómo tener una red muy estructurada localmente pero pequeña globalmente?

Estructura vs. aleatoriedad como extremos

Modelo	Clustering	Diámetro
Lattice regular G_r (cada nodo conectado a sus $2r$ vecinos más cercanos)	alto: $cl(G_r) = \frac{3r-3}{4r-2}$	alto: $diam(G_r) = \frac{N_v}{2r}$
ER(N_v, p) con $p = O(N_v^{-1})$	bajo: $O(N_v^{-1})$	bajo: $O(\log N_v)$

El modelo Watts-Strogatz

Mezcla estructura con una pizca de aleatoriedad:

1. Inicializar con un lattice regular con el clustering buscado
2. Introducir atajos: cada arista se reconecta aleatoriamente con probabilidad (pequeña) p

La reconexión interpola entre ambos extremos. En un rango intermedio de p se logra **clustering alto y diámetro bajo simultáneamente** (la “zona small-world”).

Propiedades:

- P1) Para N_v grande: $cl(G) \approx cl(G_r)(1 - p)^3$
- P2) La distribución de grados se concentra alrededor de su media $2r$ (sigue sin ser de cola pesada — ese problema lo resuelve el configuration model, no Watts-Strogatz)

Aplicaciones: dispersión de rumores/noticias (falsas), dispersión de epidemias, búsqueda de información en redes.

Modelos de Variables Latentes

Ampliamente usadas para modelar datos con observaciones parciales (ej: Hidden Markov Models, análisis factorial). Dos variantes en redes:

- **Modelos de clases latentes:** la pertenencia a una clase no observada marca la tendencia a conectarse ($\rightarrow$ SBM)
- **Modelos de vectores latentes:** la conexión depende de qué tan “cerca” están los nodos en un espacio latente ($\rightarrow$ RDPG)

Ejemplo: red de colaboración de actores en IMDb (2017-2021), $N_v = 21617$, $N_e = 73702$. Distintas tasas de conexión según dónde trabajan (Hollywood, Bollywood, Nollywood, independientes...). Ni ER ni configuration model capturan esta estructura.

Stochastic Block Models (SBM)

Grupos/comunidades $C_1, \dots, C_Q$, con tasas de conexión θ_{qr} entre/dentro de grupos.

Modelo generativo para grafo no dirigido $G(V, E)$:

- Cada vértice i pertenece independientemente a C_q con probabilidad π_q ($\pi = [\pi_1, \dots, \pi_Q]$, $\sum \pi_q = 1$)
- Para cada par $i \in C_q, j \in C_r$: $(i, j) \in E$ con probabilidad θ_{qr}

Formalmente, con $Z_{iq} = \mathbb{1}\{i \in C_q\}$: $Z_i \stackrel{iid}{\sim} \text{Multinomial}(1, \pi)$, y $A_{ij} \mid Z_i, Z_j \sim \text{Bernoulli}(\theta_{z_i, z_j})$.

- **Parámetros:** Q proporciones $\pi_q + Q(Q+1)/2$ probabilidades θ_{qr} .
- Es una mezcla de grafos ER: $P(A_{ij} = 1) = \sum_{q,r} \pi_q \pi_r \theta_{qr}$.
- Más flexible que ER/configuration para capturar estructura real, pero con temas de identificabilidad [Allman'11] y transición de fase propia [Söderberg'02-03].
- Ejemplo de matriz θ (3 comunidades, caso **asortativo**: valores en la diagonal » fuera de diagonal $\rightarrow$ más probable conectarse dentro de la propia comunidad):

$$\pi = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}, \quad \theta = \begin{bmatrix} 0.5 & 0.1 & 0.05 \\ 0.1 & 0.3 & 0.05 \\ 0.05 & 0.05 & 0.9 \end{bmatrix}$$

Nota: el SBM tampoco resuelve por sí solo el problema de la cola pesada del grado — dentro de cada bloque, el grado sigue siendo aproximadamente Binomial/concentrado (es una mezcla de ER). Para eso está el **degree-corrected SBM** (ver Extensiones, más abajo).

Grafones (graphons) y f-random graphs

Variante **no paramétrica** del SBM:

$$U_1, \dots, U_{N_v} \stackrel{iid}{\sim} \text{Unif}[0, 1], \quad A_{ij} \mid U_i = u_i, U_j = u_j \sim \text{Bernoulli}(f(u_i, u_j))$$

- **Grafón:** función simétrica y medible $f : [0, 1]^2 \rightarrow [0, 1]$. El grafo resultante es un *f-random graph*.
- U_i da la posición latente de cada nodo; $f(u_i, u_j)$ la probabilidad de conexión.
- **El SBM es un caso particular:** particionar $[0, 1]$ en Q subintervalos de largo $\pi_1, \dots, \pi_Q$, tomar el producto cartesiano (Q^2 bloques), y definir f constante a trozos con "altura" θ_{qr} en cada bloque.
- Cualquier función medible se aproxima por una constante a trozos $\rightarrow$ cualquier f-random graph se aproxima (en distribución) por un SBM, pero el número de bloques Q necesario puede ser enorme.

Plausibilidad: ¿puede un SBM aproximar cualquier grafo?

- **Cut distance** entre grafos G, G' (mismo $|V| = N_v$): $d(G, G') = \frac{1}{N_v^2} \max_{S, T} \left| \sum_{i \in S, j \in T} (A_{ij} - A'_{ij}) \right|$ — es una métrica.
- **Teorema:** para todo $\varepsilon > 0$ y todo grafo G , existe una partición en $Q \leq 2^{2/\varepsilon^2}$ clases tal que $d(G, G_P) \leq \varepsilon$ (donde G_P es la aproximación por bloques, con peso = densidad de aristas entre bloques).
- Justifica que un SBM puede aproximar cualquier grafo — pero la cota en Q puede ser gigantesca.

Extensiones de SBM

- **Degree-corrected SBM:** permite comunidades con distribución de grados amplia (cola pesada dentro de cada bloque) [Karrer-Newman'11]
- **Mixed-membership SBM:** los nodos pueden pertenecer a más de una clase [Airoldi'08]
- **SBM jerárquicos:** clustering jerárquico combinado con SBM [Clauset et al'08]

Random Dot Product Graphs (RDPG)

Espacio latente $\mathcal{X}_d \subseteq \mathbb{R}^d$ tal que $x^\top y \in [0, 1]$ para todo x, y ; distribución de producto interno F .

$$x_1, \dots, x_{N_v} \stackrel{iid}{\sim} F, \quad A_{ij} \mid x_i, x_j \sim \text{Bernoulli}(x_i^\top x_j)$$

Posiciones $X = [x_1, \dots, x_{N_v}]^\top \in \mathbb{R}^{N_v \times d}$.

Conexión con otros modelos

- **ER(n,p)** = RDPG con $d = 1$, $\mathcal{X}_d = \{\sqrt{p}\}$
- **SBM** = RDPG con F tal que $P(X = x_q) = \pi_q$ y $x_q^\top x_r = \theta_{qr} \rightarrow$ **los RDPG son al menos tan expresivos como los SBM**
- Caso especial de modelos de posición latente [Hoff'02]: $A_{ij} \sim \text{Bernoulli}(\kappa(x_i, x_j))$; el RDPG aproxima cualquier κ con d suficientemente grande [Tang'13]

Estimación: Adjacency Spectral Embedding (ASE)

- MLE es la idea natural pero inviable para N_v grande.
- $P_{ij} = P((i, j) \in E)$; el modelo dice $P = XX^\top$, y A es una realización ruidosa de P ($E[A] = P$) $\rightarrow$ mínimos cuadrados: $\hat{X}_{LS} = \arg \min_X \|XX^\top - A\|_F^2$
- A real y simétrica: $A = U\Lambda U^\top$ (autovalores $\lambda_1 \geq \dots \geq \lambda_{N_v}$). Tomando los d autovectores de mayores autovalores positivos: $\hat{\Lambda} = \text{diag}(\lambda_1^+, \dots, \lambda_d^+)$, $\hat{U} = [u_1, \dots, u_d]$.
- **ASE**: $\hat{X}_{LS} = \hat{U}\hat{\Lambda}^{1/2}$ (mejor aproximación de rango d semidefinida positiva de A)
- **¿Es única la solución?** No — el producto interno es invariante a rotaciones ($XW(WX^\top)^\top = XX^\top$ si $WW^\top = I$). El embedding es identificable **módulo rotaciones**.
- Elegir d : buscar un “codo” en el gráfico de autovalores (scree plot).

Interpretabilidad

- Ejemplo Zachary's karate club ($N_v = 34$): con $d = 2$, el administrador y el instructor del club quedan con embeddings **ortogonales**.
- **Alineación del vector** $\rightarrow$ afinidad/comunidad entre nodos
- **Magnitud del vector** $\rightarrow$ conectividad (grado) del nodo
- Ejemplo grafo bipartito de votación del Senado de EEUU: polarización clara en 2019, no tanto en 1977.

Extensiones y aplicación: online change point detection

- **Generalized RDPG** [Rubin-Delanchy'17]: cuando A no es semidefinida positiva
- Extensiones a grafos pesados y/o dirigidos [Marengo et al]
- **Detección de cambios online**: entrenar con m grafos “limpios” (sin cambios); monitorear la suma acumulada $S[m, k] = \sum_{t=m+1}^{m+k} (\hat{X}\hat{X}^\top - A[t])$; bajo la hipótesis nula (sin cambio), $\psi[m, k] = \|S[m, k]\|^2$ tiene una distribución χ^2 generalizada. Liviano: costo $O(N^2)$.
 - Ejemplo real: red inalámbrica del Plan Ceibal (6 APs), detecta que un AP fue movido físicamente — se explica el cambio vía la interpretabilidad del ASE.

Grafos Geométricos (RGG)

Espacio vectorial con distancia $d(u, v)$ y subconjunto B :

$$x_1, \dots, x_{N_v} \stackrel{iid}{\sim} \text{Unif}(B), \quad A_{ij} = \mathbb{1}(d(x_i, x_j) < r)$$

Ejemplo clásico: $\mathbb{R}^2$ con norma Euclídea, B = disco de radio R .

Propiedades (suponiendo $r \ll R$):

- P1) Grado promedio: $\bar{D} = \rho \pi r^2$, con densidad $\rho = N_v / (\pi R^2)$
- P2) Distribución de grados **Poisson**: $P(D_i = d) = \bar{D}^d e^{-\bar{D}} / d!$ → buen clustering, pero **cola cae muy rápido (no es cola pesada)** — mismo problema estructural que ER, no captura hubs.

Solución: no usar geometría Euclídea → geometría hiperbólica (que, como se ve abajo, sí produce cola pesada / power-law).

Geometría Hiperbólica y Curvatura

- Círculo de radio r en esfera de radio R : $C(r) = 2\pi R \sin(r/R) < 2\pi r$ — **curvatura positiva** → perímetros/áreas *menores* que en el plano.
- **Curvatura negativa:** perímetros/áreas *mayores* que en el plano (requiere puntos silla).
- **Pseudoesfera** (curvatura negativa constante): mapa conforme → métrica $ds' = R \frac{\sqrt{dx^2 + dy^2}}{y}$ — el **modelo del semiplano de Poincaré**.
- Otras representaciones equivalentes: **disco de Poincaré-Beltrami** (el círculo unitario es el horizonte) y el **hiperboloide** con producto de Minkowski.

Hyperbolic Random Graphs

Puntos uniformes en un círculo de radio R , pero en espacio **hiperbólico**; cada nodo se conecta con los que están a distancia R o menos. El volumen crece **exponencialmente** con la distancia: $C(r) \approx e^r$, $A(r) \approx e^r$.

Propiedades:

- P1) Grado medio $\bar{D} \propto N_v e^{-R/2}$ → escalando R logarítmicamente ($R = 2 \log(N_v/\nu)$) se obtiene un grafo esparso con grado medio finito, **independiente de N_v**
- P2) Distribución de grados **power-law con exponente 3** (cola pesada — a diferencia del RGG euclídeo de arriba) — otros exponentes posibles con distribución no uniforme del radio
- P3) Clustering controlable con un parámetro de “temperatura” T en la versión soft: $p_{ij} = \frac{1}{1 + e^{\frac{1}{2T}(d(x_i, x_j) - R)}}$
- P4) También produce el efecto small-world

La curvatura negativa es la única que permite un modelo con todas las propiedades “realistas” a la vez (small-world + clustering alto + cola pesada / power-law en el grado). Es el punto de llegada de toda la clase: cada modelo anterior lograba una o dos de estas propiedades, pero ninguno las tres juntas.

Métodos de embedding: basados en probabilidad (ajustan R, T maximizando verosimilitud, ej. *Mercator*) o basados en distancia (ubican los embeddings para reflejar distancias conocidas, ej. *Hydra*). Nunca hay forma cerrada → gradientes/heurísticas/aproximaciones. Librería: github.com/CicadaUY/hypeGRL.

Resumen y moralejas (tabla del profesor — repaso rápido)

Modelo	Idea nueva	Qué agrega
Erdős–Rényi	aristas i.i.d. (modelo nulo)	línea base: diámetro chico, pero clustering → 0
Configuration model	fijar la distribución de grados	cualquier distribución de grados (ej. cola pesada)
Watts–Strogatz	lattice + una pizca de azar	clustering alto y small-world a la vez
SBM / grafones	clases latentes (y su límite)	comunidades / heterogeneidad

Modelo	Idea nueva	Qué agrega
RDPG	vectores latentes ($x_i^\top x_j$)	embeddings interpretables (ASE)
Hiperbólicos	geometría del espacio latente	todas las propiedades a la vez (incluida la cola pesada)

Moralejas:

- No hay un “mejor” modelo: se elige según el uso (explicar / testear / predecir / generar)
- Más realismo → más parámetros y mayor costo de inferencia/generación; rara vez hay forma cerrada (de ahí gradientes, heurísticas, aproximaciones)
- SBM, grafones, RDPG e hiperbólicos son todos de **variables latentes**: lo que cambia es la geometría del espacio latente
- Esa geometría decide qué propiedades emergen: **la curvatura negativa es la que las produce todas juntas**

ewpage

Clase 3 – Inferencia de grafos

Apunte de clase, completado y verificado contra [inferencia.pdf](#) (56 diapositivas, Marcelo Fiori). Lo agregado a partir de las diapositivas está marcado con > .

Objetivo

Inferir un grafo a partir de datos (señales observadas en los nodos). Se busca el grafo que **mejor explica la señal** — ¿cómo usar GSP (Graph Signal Processing) para inferir la topología del grafo?

Ejemplo motivador: en *network neuroscience*, inferir la red funcional del cerebro a partir de señales de fMRI (qué regiones se comunican entre sí).

La mayoría de los trabajos de GSP asumen un grafo **conocido** y estudian cómo afecta a las señales (tiene sentido en redes físicas, donde los enlaces son observables). Acá se hace el **camino inverso**: usar GSP para inferir la topología. Motivo práctico: mantener actualizada la topología real de una red suele ser difícil (tamaño, reconfiguración, privacidad, seguridad).

Objetivo formal: recuperar una *red latente*, o una representación en grafo de los datos.

Los 3 problemas de inferencia de grafos

La pregunta general en todos los casos: ¿hay o no hay una arista entre dos nodos? ¿con qué peso?

Formalización común (marco de [Kolaczyk’09]): dado un grafo que no se observa (o se observa solo parcialmente), se busca inferirlo a partir de mediciones de señal en los nodos y, eventualmente, el estado de algunas aristas ya conocidas. Formularlo así permite aprovechar herramientas estadísticas ya conocidas: identificabilidad, consistencia, robustez, complejidad computacional.

1. **Predicción de enlaces** (presentado clase anterior): se conoce la señal completa y el estado de las aristas solo para un subconjunto de pares de vértices; se quieren inferir las aristas que faltan (el resto de los pares).
2. **Association network inference** (tema de hoy): se tienen señales en los nodos pero no se conoce ninguna arista — se asume que la conexión entre dos nodos está definida por un nivel de asociación no trivial entre sus señales, y se quiere inferir el estado de arista para **todos** los pares.
3. **Tomographic network topology inference**: se observa la señal solo en algunos vértices ubicados en el “perímetro” del grafo (los accesibles); el objetivo es inferir vértices y aristas del “interior”, la parte no observada. Es el caso más difícil de los tres.

Association Networks

Definición: los vértices quedan unidos por una arista si hay un nivel suficiente de “asociación” entre los atributos de cada par de vértices.

Ejemplos de dominio: *gene-regulatory networks*, *neuro-functional connectivity networks*.

Datos: matriz donde las filas representan los nodos y las columnas la señal x a lo largo del tiempo.

Preguntas centrales:

- ¿El grafo inferido es único? $\rightarrow$ depende de las condiciones.
- ¿Cuántas mediciones hay que hacer del experimento? Se quiere estimar las aristas dado un número de mediciones. Cuanto más larga sea la señal, mejor la estimación.

Association network inference (definición formal del problema): dada una señal de atributos observados en los nodos, y una similaridad definida por el usuario $\text{sim}(i, j) = f(x_i, x_j)$ que especifica las aristas, el problema es **inferir los valores no triviales de esa similaridad** a partir de observaciones i.i.d. — ya que el estado real de las aristas (los valores de sim) no es observable directamente.

Hay **tres categorías de decisiones** a tomar para resolver este problema:

Decisión	Opciones típicas
Elección de sim	correlación, correlación parcial, información mutua
Elección de técnica de inferencia	test de hipótesis, regresión, ad hoc
Elección de parámetros	umbrales de test, nivel de significancia, regularización

Redes de correlación

Idea: si dos nodos están correlacionados, sus señales son similares.

- Se calcula la **correlación** (de Pearson) entre cada par de señales temporales.
- **Grafo de correlación:** se pone una arista entre dos nodos si la correlación es distinta de cero.
- Association network inference $\iff$ inferencia de correlaciones no nulas.

Test de hipótesis: $H_0 : \rho_{ij} = 0$ (no hay correlación) versus $H_1 : \rho_{ij} \neq 0$ (hay correlación). Habría que hacer N^2 tests de hipótesis, pero por simetría alcanza con $N(N-1)/2$ — de todas formas resulta costoso computacionalmente.

¿Qué es un test de hipótesis? Es un procedimiento para decidir, a partir de los datos, entre dos hipótesis: la **hipótesis nula** H_0 (por convención, la situación “por defecto” o de ausencia de efecto — acá, “no hay correlación”) y la **hipótesis alternativa** H_1 (hay efecto). Se calcula un estadístico a partir de los datos y se **rechaza** H_0 si ese estadístico resulta demasiado extremo como para ser plausible bajo H_0 . El **nivel de significancia** α es la probabilidad que se tolera de rechazar H_0 por error (falso positivo) cuando en realidad era cierta.

Estadísticos usados para el test (cómo se hace la cuenta):

- Correlaciones empíricas.
- **Transformada de Fisher:** lleva la correlación empírica a una variable con distribución (aproximadamente) normal bajo H_0 , lo que simplifica controlar la significancia del test. Se rechaza H_0 si el estadístico supera un umbral asociado a α .

Testeo múltiple: al hacer muchos tests de hipótesis a la vez (uno por par de nodos) aparece el problema de testeo múltiple en grafos. > Ejemplo: con $N = 100$ nodos y $\alpha = 0,05$, aunque el grafo real fuera vacío se esperarían **~250 aristas espúreas** solo por azar (de los 4950 pares posibles) — para un grafo grande, el número de falsos positivos puede ser considerable.

Se corrige con **False Discovery Rate (FDR)**: en vez de controlar el error por test individual, se controla la fracción esperada de aristas falsas sobre el total de aristas declaradas. > Método práctico (Benjamini-Hochberg): se ordenan

los p -valores de todos los tests, y se declaran arista todos los pares cuyo p -valor esté por debajo de un umbral que crece según su posición en el ranking (no un umbral fijo como en el testeo individual).

Ejemplo aplicado (E. coli, genes *aroG*/*tyrR*/*lrp*): *aroG* está regulado por *tyrR* pero no por *lrp* (ground truth conocido). Sin embargo, ambos muestran correlación alta y p -valor chico con *aroG* — un caso concreto de cómo la correlación simple puede dar **falsos positivos por factor de confusión**, motivando el uso de correlación parcial.

Correlaciones parciales

Motivación: > > “**correlación no implica causalidad**” — un par de nodos puede estar correlacionado porque ambos están influenciados por un tercer nodo, no porque estén conectados directamente entre sí. (Para estos problemas los grafos no tienen self-loops.)

Las **correlaciones parciales** capturan mejor la influencia directa entre nodos, descontando el efecto de terceros nodos. > Se calculan a partir de los bloques de la matriz de covarianza (covarianza y varianzas condicionales, eliminando el efecto lineal de los nodos sobre los que se condiciona).

Redes de correlaciones parciales: se construyen igual que las redes de correlación, pero usando correlación parcial en vez de correlación simple — con su propio test de hipótesis análogo.

Case study: regulación génica en E. coli. - Contexto: los genes que controlan la expresión de otros son *transcription factors* (TFs); los controlados son *targets*; la regulación puede ser activación o represión. - Datos: 4.345 genes bajo 445 condiciones experimentales, 153 TFs conocidos, pares TF/target de referencia (base RegulonDB). - Se comparan 3 métodos: (1) correlación de Pearson simple, (2) correlación parcial condicionando a un solo TF por vez, (3) correlación parcial completa condicionando simultáneamente a los 152 TFs restantes. - Resultado: el método 1 es el peor, pero **ninguno funciona muy bien** — la correlación no es un indicador fuerte de regulación en estos datos (alta precisión pero muy bajo recall). - Aun así, sirve para *generar hipótesis*: para el TF *lrp* se encontraron 11 interacciones candidatas, 10 confirmadas experimentalmente y 5 nuevas no reportadas antes.

Undirected Gaussian Graphical Models

Si se asume que las variables $\{x_i\}$ tienen distribución gaussiana multivariada, vale un **teorema clave**: la correlación parcial entre x_i y x_j , condicionando a *todos* los demás nodos, es cero **si y solo si** x_i y x_j son condicionalmente independientes dado el resto. Es decir, bajo el supuesto gaussiano, “correlación parcial (completa) nula” y “sin relación directa entre esos dos nodos” son exactamente lo mismo.

Esto define el **grafo de independencia condicional**: hay arista entre i y j si y solo si esa correlación parcial completa es distinta de cero — un caso particular y muy usado de red de correlación parcial. También se lo conoce como **Gaussian Markov Random Field (GMRF)**.

Covariance selection: se define la matriz de precisión $\Theta = \Sigma^{-1}$. Resultado clave: bajo el modelo GMRF, una entrada no nula en Θ equivale exactamente a una arista en el grafo. El problema de inferir G usando esta equivalencia se llama *covariance selection* [Dempster’74]. Los métodos clásicos testean cada par por separado y en general no escalan bien: cuando el número de observaciones P es mucho menor que el número de nodos N , estimar $\hat{\Sigma}$ de forma confiable es difícil.

GMRFs con restricción de Laplaciano: una variante impone que Θ sea directamente la Laplaciana del grafo. Esta formulación **favorece grafos sobre los que las señales son suaves** — es el puente conceptual hacia la sección siguiente.

Digresión: sparsity y norma ℓ_1

- La norma ℓ_1 induce **esparsidad** (estimador **Lasso** [Tibshirani’94]): una ventaja de esta norma es que hace que algunas coordenadas se anulen exactamente.

- **Graphical Lasso** [Yuan-Lin'07]: estima toda la matriz de precisión en conjunto, con una penalización ℓ_1 que promueve esparsidad. Efectivo cuando $P \ll N$; da modelos interpretables. > Existen métodos escalables basados en *coordinate-descent*.

Covariance selection meets linear regression

Objetivo: mejorar la explicación del método / chequear el algoritmo, estimando la matriz por partes usando combinaciones lineales y viendo qué nodos aportan más.

Formalización: el valor esperado de x_i condicionado al resto de las variables es una función **lineal** de esas variables, y los coeficientes de esa regresión están directamente relacionados con las entradas de Θ — el soporte (coeficientes no nulos) de esa regresión coincide exactamente con la vecindad del nodo en el grafo. Esto justifica estimar el grafo vía regresión en vez de estimar toda la matriz de precisión.

- Método práctico: se toma la primera fila, se hace una regresión, y se conservan los pocos coeficientes más significativos.

Neighborhood-based sparse regression (NBR): se resuelve un problema Lasso independiente por cada nodo (regresión de ese nodo contra todos los demás). > Como no hay garantía de que la relación salga simétrica, se usan reglas **OR** o **AND** para decidir si declarar una arista cuando solo uno de los dos nodos “ve” al otro como vecino.

Los tres enfoques vistos — testear correlaciones parciales, covariance selection (vía Θ), y neighborhood-based regression (vía β) — son **matemáticamente equivalentes**, solo cambia el camino de cálculo.

Comparación NBR vs. Graphical Lasso:

	NBR	Graphical Lasso
Optimiza	verosimilitud condicional por nodo (independientes)	verosimilitud global (regularizada)
Fuerza $\Theta \succeq 0$	No	Sí
Velocidad	más rápido, paralelizable	más lento
Eficiencia estadística	menor	mayor (usa mejor la info conjunta)

NBR se puede extender a variables discretas o mixtas (*Ising-model selection*).

Estos modelos (Gaussian graphical models, graphical Lasso, covariance selection) son para **variables aleatorias Gaussianas**.

Aprendiendo grafos a partir de observaciones de señales suaves

Motivación: se busca un grafo sobre el cual las señales admitan regularidad, por ejemplo para predicción por vecino más cercano (*graph smoothing*) o aprendizaje semi-supervisado. Muchos datos de redes reales son suaves porque la formación de la red suele basarse en homofilia o proximidad en un espacio latente (nodos parecidos tienden a conectarse, y por ende a tener valores de señal parecidos).

Planteo formal del problema: dadas observaciones X , identificar un grafo G tal que esas señales sean *suaves* en G . El criterio de suavidad es la **energía de Dirichlet**, $TV(x) = x^\top Lx$: cuanto más chica, más suave la señal respecto al grafo.

Ejemplos de aplicación: predicción de función de proteínas en levadura (proteínas conectadas tienden a compartir función); red de colaboración entre abogados (colaboran más entre pares del mismo tipo de práctica legal).

Recordando el ejemplo de la primera clase: a medida que avanza la señal, es menos suave. Frecuencia grande $\rightarrow$ coeficiente chico.

Graph Signal Processing (GSP) y Graph Fourier Transform (GFT): se usan los vectores propios de la matriz del grafo (adyacencia o Laplaciana) como “base frecuencial”, igual que en procesamiento de señales

clásico se explota la estructura temporal (un ciclo temporal da lugar a la Transformada de Fourier clásica). La GFT de una señal es su proyección en esa base; la GFT inversa reconstruye la señal original.

Modos frecuenciales de la Laplaciana (esto es a lo que se refería el ejemplo de “Diapo 41”): la Total Variation $TV(x) = x^T Lx$ mide la suavidad de una señal respecto al grafo. Para los vectores propios de la Laplaciana, ese valor coincide exactamente con el autovalor asociado — por eso los autovalores de L se interpretan como “frecuencias”: autovalor chico = modo suave (frecuencia baja), autovalor grande = modo poco suave (frecuencia alta).

Ahora se infiere el grafo a partir de la **matriz Laplaciana**: se aprende un grafo a partir de una señal suave.

Modelo de análisis factorial basado en la Laplaciana [Dong et al’16]: se modela la señal observada como combinación de los modos de Fourier del grafo más ruido, dando más peso a los coeficientes asociados a autovalores chicos (baja frecuencia) — el modelo favorece explícitamente señales “pasa-bajo” (suaves).

Inferencia como denoising: buscar el grafo se puede plantear como buscar simultáneamente la Laplaciana L y una versión “limpia” (sin ruido) de la señal, penalizando la falta de suavidad de esa versión limpia respecto a L .

Formulación y algoritmo: hay que definir cuánta importancia se le da a la suavidad y cuánta a la esparsidad. > Concretamente, se combina un término de ajuste a los datos, un término de suavidad y un término de esparsidad de aristas, con restricciones que evitan la solución trivial de grafo vacío. El problema no es conjuntamente convexo en L y en la señal limpia, pero sí es *bi-convexo*, lo que permite resolverlo por **minimización alternada**: fijando la señal limpia se optimiza L ; fijando L , la señal limpia óptima tiene solución cerrada y equivale a aplicar a los datos un filtro pasa-bajos.

Impacto de los parámetros: más peso a la esparsidad da menos aristas; cuando hay poco ruido, el cociente entre el parámetro de suavidad y el de esparsidad determina la calidad del grafo recuperado.

Ejemplo aplicado (temperaturas en Suiza): con 89 estaciones meteorológicas, usar directamente distancia geográfica no captura bien la similitud de temperatura (por diferencias de altura). Aprendiendo el grafo desde las temperaturas mismas y aplicando *spectral clustering* sobre él aparecen dos clusters sensatos (estaciones altas vs. bajas), cosa que *k-means* directo sobre las temperaturas no logra.

Signal smoothness meets edge sparsity

Alternativa a usar la Laplaciana directamente [Kalofolias’16]: existe una relación directa entre la suavidad total de las señales y la norma ℓ_1 de la matriz de adyacencia ponderada por las distancias entre nodos. En otras palabras, minimizar la falta de suavidad equivale a favorecer aristas entre nodos con señales parecidas (distancia chica) y penalizar las demás. Esto permite reformular el problema directamente en términos de la matriz de adyacencia A en vez de L , lo cual simplifica las restricciones (quedan desacopladas).

El modelo agrega una barrera logarítmica que evita que algún nodo quede con grado cero, y un término que penaliza pesos grandes para controlar la densidad del grafo. Se resuelve con un método primal-dual paralelizable.

Ejemplo aplicado (dígitos USPS): aprendizaje del grafo para clustering espectral en 10 clases de dígitos manuscritos — este enfoque resulta más robusto a la densidad elegida para el grafo (no deja nodos aislados) que un grafo de vecinos más cercanos (k-NN) clásico.

Aprendizaje de grafos seleccionando aristas

Alternativa [Chepuri et al’17]: en vez de optimizar sobre pesos continuos, se representa la topología con un vector binario que indica qué aristas están presentes, permitiendo fijar directamente el número exacto de aristas deseado (K). Sin ruido, la solución es simple: calcular un “score” de suavidad para cada arista candidata y quedarse con las K de menor score. Existe una versión más realista con ruido en las mediciones.

Comparación de este enfoque:

A favor	En contra
Controla directamente la cantidad de aristas	No garantiza que el grafo quede conectado
Algoritmo simple sin ruido	No permite pesos en las aristas (todas iguales)
No hace falta imponer restricciones extra para tener una Laplaciana válida	

Case study: clasificación con grafos discriminativos

Si se tienen señales etiquetadas de distintas clases, se puede aprender **un grafo distinto por clase** (asumiendo que las señales de cada clase son suaves respecto a un grafo propio), agregando un término que penaliza que ese grafo también favorezca la suavidad de señales de *otras* clases (discriminabilidad) [Saboksayr et al'21]. Para clasificar una señal nueva, se la proyecta usando la base de Fourier de cada grafo aprendido y se elige la clase cuya proyección concentra más energía en las frecuencias bajas.

Aplicación real: reconocimiento de emociones a partir de señales EEG (dataset DEAP, 32 personas, 32 canales), clasificando valencia emocional alta vs. baja — accuracy promedio de 92,7%. Los grafos aprendidos por clase muestran diferencias interpretables en la conectividad del lóbulo frontal según la intensidad emocional.

* Posibles puntos de examen

- Diferencia entre los 3 problemas canónicos de inferencia de topología: qué se observa y qué se busca en cada uno (predicción de enlaces / association network inference / tomographic).
- Qué es un test de hipótesis, y cómo se define H_0 en el contexto de redes de correlación.
- Qué es el problema de testeo múltiple en grafos y para qué sirve el FDR.
- “Correlación no implica causalidad”: por qué se necesita correlación parcial y no alcanza con correlación simple.
- Bajo qué supuesto (gaussiano) la correlación parcial completa nula equivale a independencia condicional (GMRF).
- Relación entre la matriz de precisión Θ y las aristas del grafo (covariance selection).
- Diferencias entre Graphical Lasso y neighborhood-based regression (velocidad vs. eficiencia estadística).
- Qué mide la Total Variation / energía de Dirichlet, y qué representan los autovalores de la Laplaciana (frecuencias del grafo).
- Qué términos combina la función objetivo para aprender un grafo desde señales suaves (ajuste + suavidad + esparsidad).
- Diferencia entre parametrizar el aprendizaje del grafo en L vs. en A (enfoque de Kalofolias).
- Ventajas y desventajas del enfoque de selección de aristas por cardinalidad fija.

ewpage

Clase 4 – Graph Neural Networks (GNNs)

Intro

- Algunos ejemplos de motivación > (de diapositivas):
 - **Clasificación de nodos:** ¿a qué área pertenece un nuevo paper de arXiv? Se usa el grafo de citas (papers ya categorizados citados por el nuevo).
 - **Predicción de aristas / datos faltantes:** estimar el canal (RSSI) entre un dispositivo y varios *access points* a partir de mediciones parciales — equivale a “completar” enlaces faltantes en el grafo.
 - **Clasificación de grafos:** localización indoor de un usuario usando únicamente el grafo de mediciones RSSI a distintos APs.

Motivación: Empirical Risk Minimization (ERM)

- Tenemos: conjunto de entrenamiento, una clase de funciones (acá es donde entran las GNN), y una función de pérdida (mide la bondad de ajuste).
- Buscamos, dentro de la familia de funciones, la que minimiza la función de pérdida.
- ¿Qué es un dato en este contexto?
 - Un grafo
 - Señal en los nodos
 - Atributos en las aristas

Representando datos en grafos

- Se toma una decisión: ponerle un orden a los nodos, y por ende también a la señal (es arbitrario).
- Se busca que la función **no dependa del orden** elegido, y que sea aplicable a grafos de distinto tamaño (poder operar sobre grafos distintos con la misma función).

Convoluciones en grafos

- Referencia: Convolutional Neural Networks (CNNs).
- **CNN**: toma una matriz y genera otra usando la convolución.
- En señales en tiempo discreto, la cercanía (entre instantes) contiene información.

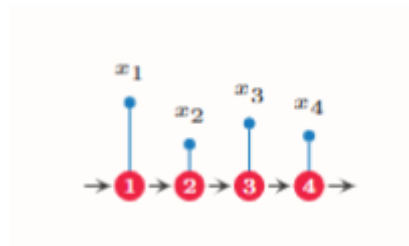


Figure 1: alt text

- Hace falta un buen orden para que la sucesión sea suave. Como es una serie de tiempo, el grafo que la representa es una línea.
- Se fija el orden de los nodos $\rightarrow$ permutaciones $\rightarrow$ queda definida la matriz S genérica (**Graph Shift Operator**).
- **Graph convolution**: combinación lineal de versiones desplazadas de la señal (el grafo es el mismo, solo se desplaza la señal hacia la derecha de a un paso; esto se logra multiplicando por un operador).

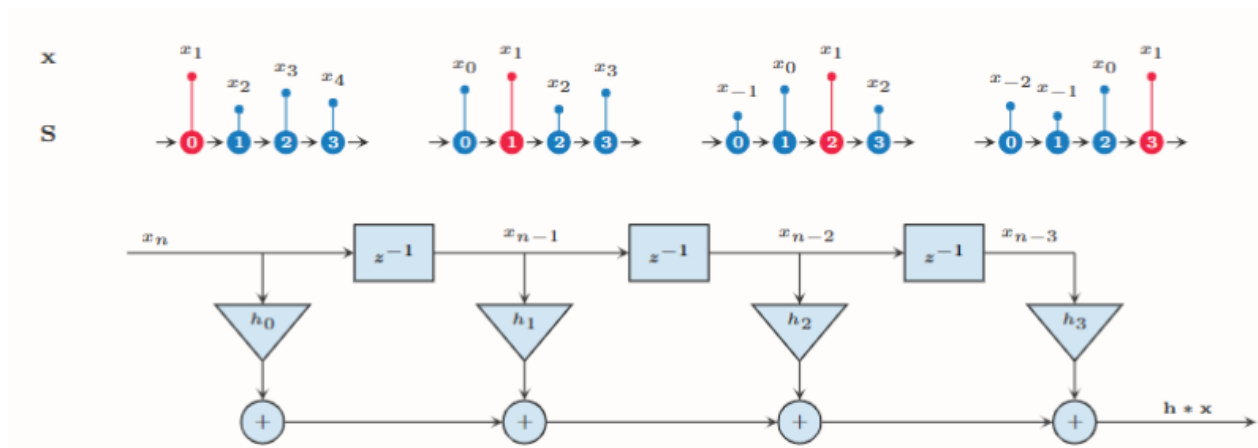


Figure 2: alt text

- K se elige (expande la “máscara” del filtro).
- (de diapositivas) La cadena “**Graph Convolution** => **Graph Signal Processing** => **Graph Filtering**” no expresa una relación causal entre tres conceptos distintos: son tres nombres / ángulos de la misma idea (análogo a “Convolution => Signal Processing => Filtering” en el procesamiento de señales clásico).

De información local a global

- Se puede operar el mismo filtro en distintos grafos.

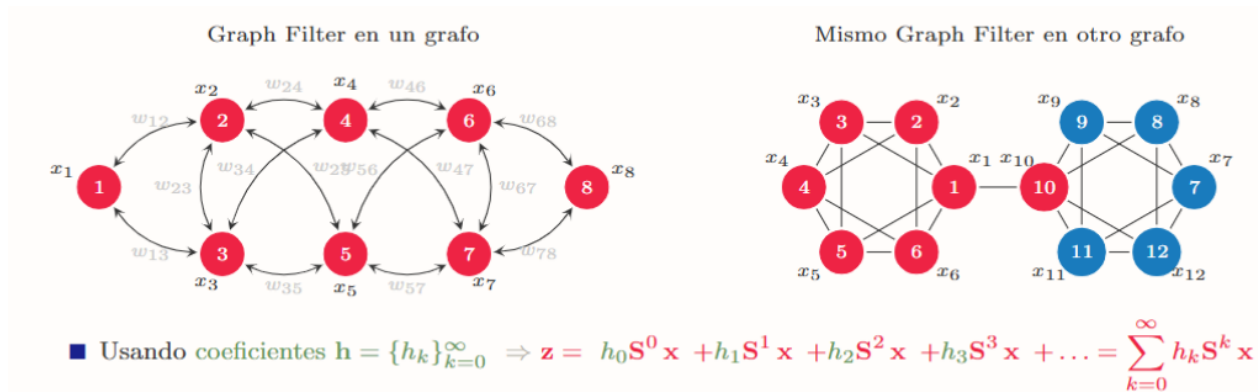


Figure 3: alt text

- En principio, la familia de funciones definida así cumple con lo que se buscaba (independencia del orden, aplicable a grafos de distinto tamaño).

Hasta acá se vieron filtros. Sigue el aprendizaje.

GNN: Construcción

- Esquema: Señal $\rightarrow$ Filtro $\rightarrow$ Nueva señal.
- Aprendiendo usando un **Graph Perceptron**.
- La salida da un parámetro H (tensor) — se va a querer elegir la cantidad de h 's.
- Un graph filter puro tiene expresividad limitada: solo puede aprender funciones **lineales**. El **Graph Perceptron** agrega una no linealidad puntual σ a la salida del filtro; apilando varios perceptrones (capas) se aumenta aún más la expresividad (eso es la GNN).
- La única diferencia entre una GNN y un graph filter es esa no linealidad punto a punto — pero en la práctica las GNN funcionan mucho mejor (la explicación formal de por qué viene más adelante, con el análisis de estabilidad ante perturbaciones).
- (de diapositivas) **Ventajas de la GNN** (resuelve el punto pendiente):
 - **Transferencia entre grafos:** el GSO S se puede reinterpretar como una *entrada* (no como un parámetro fijo) $\rightarrow$ una GNN ya entrenada (el tensor H^*) se puede aplicar a otro grafo sin reentrenar.
 - **Menos parámetros, generaliza mejor:** frente a una red “plana” que viera toda la matriz de adyacencia como una entrada más, la GNN comparte los mismos parámetros H en todos los nodos (no dependen de N) y aprovecha la estructura local del grafo.
 - **Aplica a grafos de distinto tamaño**, a diferencia de una red neuronal genérica atada a un N fijo.

Múltiples features

(visto muy rápido en el apunte)

- En las librerías se habla poco de “convolución”; se habla más de **message passing**.
- Ambos “son lo mismo”.
- *(de diapositivas)* Formalización: una señal con múltiples features es una matriz X de tamaño N (nodos) $\times F$ (features) — cada fila son los features de un nodo, cada columna es una graph signal individual. La convolución se generaliza a

$$Y = \sum_k S^k X H_k$$

donde multiplicar por S desplaza cada feature (columna) por separado, y multiplicar por H combina los features **dentro de cada nodo** (no mezcla nodos entre sí).

- *(de diapositivas)* Hiperparámetros de una GNN multi-feature (afectan la cantidad de parámetros a aprender):
 - L : número de capas.
 - K_ℓ : orden del filtro en la capa ℓ .
 - F_ℓ : cantidad de features en la capa ℓ .

Message Passing > *(de diapositivas)*

- Las librerías (ej. PyTorch Geometric) no hablan de “convolución” sino de **message passing** — es el mismo concepto, con $K_\ell = 1$ por capa: cada capa agrega información solo de los vecinos inmediatos, y el alcance mayor se logra apilando varias capas.
- Varias arquitecturas conocidas son casos particulares de este mismo framework, con distintas elecciones de S y H : Spectral GCNNs, ChebNets, Diffusion CNNs, entre otras.

Entrenando una GNN

- Una vez construida la GNN, se quiere llegar a un vector que represente algo.
- Aprender $\Leftrightarrow$ minimizar la pérdida.
- *(de diapositivas)* Ejemplos de función de pérdida: **L2** para regresión, **cross-entropy** para clasificación.

Predicción a nivel de nodo

- Ejemplo: decidir si un usuario de una red social es un bot.
- *(de diapositivas, resuelve punto pendiente)* Durante el **entrenamiento** solo participan los nodos del training set (se usa una versión recortada del S); al **predecir**, se usa el S completo (todos los nodos).

Predicción de aristas > *(de diapositivas)*

- Se usa la salida de la GNN en los **dos nodos extremos** de una arista para estimar si esa arista existe (o cuánto vale su peso).
- Ejemplos: recomendación de contenido a usuarios, completar datos faltantes en una base de datos relacional, estimar la atenuación en una red inalámbrica.

Clasificar grafos

- Si la identidad de los nodos **no** importa: > *(de diapositivas)* se puede resumir/agregar el grafo de forma simétrica (agregación permutation-invariant) — es el caso donde una GNN estándar rinde bien.
- Si la identidad de los nodos **sí** importa, una GNN “no es lo mejor”: > *(de diapositivas)* alternativas son **Edge-Variant GNNs**, **Feature Augmentation** y **Attention**.

Permutation equivariance

- *(de diapositivas)* Una GNN es equivariante a permutaciones si, al reordenar los nodos del grafo (y su señal), la salida se reordena exactamente igual, sin cambiar de valores — es el análogo, en grafos, de la invarianza a traslaciones de las CNN.
- Por qué es valiosa: permite aprovechar simetrías internas del grafo y que la GNN generalice entre distintos órdenes/etiquetados de nodos sin reentrenar.
- No siempre es deseable: si la identidad/posición de un nodo debería importar para la tarea, forzar la equivariance es una limitación (mismo caso que en “Clasificar grafos” arriba — alternativas: Edge-Variant GNNs, Feature Augmentation, Attention).

Perturbaciones

- Fourier *(mencionado, sin desarrollo completo en el apunte)*
 - *(de diapositivas)* Motivación: a diferencia de las convoluciones temporales clásicas, no hay una intuición directa sobre la estabilidad de las graph convolutions frente a cambios en el grafo. Para estudiar eso se recurre a la **Graph Fourier Transform**: permite ver qué le pasa a un filtro cuando el grafo se perturba levemente.
 - *(El desarrollo completo — transformada, respuesta en frecuencia, por qué las no linealidades ayudan a la estabilidad — sigue más adelante en las diapositivas; probablemente continúe en la próxima clase.)*
-

* Posibles puntos de examen

- Diferencia entre un **graph filter** y un **Graph Perceptron** (la no linealidad punto a punto) y por qué la segunda es más expresiva.
- Qué es el **Graph Shift Operator** S y qué rol cumple (matriz que codifica al grafo, ej. adyacencia).
- **Message passing** $\equiv$ **graph convolution** con $K = 1$ por capa; el alcance mayor se logra apilando capas.
- Por qué durante el **entrenamiento** de una GNN para predicción a nivel de nodo solo participan los nodos del training set (S recortada), mientras que en **inferencia** se usa el grafo completo.
- Los **tres problemas canónicos** en GNN (nodo / arista / grafo) y qué salida de la red se usa en cada caso.
- **Ventajas de una GNN** frente a una red neuronal genérica: comparte parámetros (no dependen de N), es transferible entre grafos, aplica a grafos de tamaño variable.
- Definición de **permutation equivariance** y su analogía con la invarianza a traslaciones de las CNN.
- Cuándo la permutation equivariance **no** es una buena idea, y qué alternativas existen (Edge-Variant GNNs, Feature Augmentation, Attention).
- Motivo para introducir la **Graph Fourier Transform**: estudiar la estabilidad de un filtro ante perturbaciones del grafo.

[!] Puntos poco claros / a revisar

- **Resueltos con las diapositivas**: la relación “Graph Convolution $\Rightarrow$ GSP $\Rightarrow$ Graph Filtering” (son sinónimos, no una cadena causal); la “Ventaja GNN” que había quedado sin completar; la duda de si se ocultan o no las etiquetas al entrenar (se recorta el grafo a los nodos de training, no se ocultan etiquetas dentro del mismo grafo); el contenido de “Predicción de aristas”; la frase incompleta de “Clasificar grafos”.
- **“Sección 11, algoritmo”**: corresponde a la sección “Expresividad” del deck (test de Weisfeiler-Lehman para la capacidad de una GNN de distinguir nodos/grafos). Queda bastante más adelante en las diapositivas (después de Perturbaciones y Wi-Fi Indoor Positioning) — no parece ser parte de esta clase, sino de una siguiente.
- **Permutation equivariance y Perturbaciones**: se agregó la idea conceptual base, pero el desarrollo matemático completo (demostración de equivariancia, transformada de Fourier en grafos, análisis de estabilidad) continúa en las diapositivas más allá de donde llegó esta clase — probablemente se retome en la clase siguiente.